

Task 5 — AI-Powered Document RAG Chatbot Dashboard

AI Internship Program — Practical Evaluation Project

Purpose of this task: Build a complete AI-powered Document Question Answering (RAG) application. Users upload documents (PDF/DOCX/TXT); the system extracts and organizes their content, converts it into searchable embeddings, and stores it in a vector database. Users then interact with that knowledge through a chatbot: when they ask a question, the system retrieves the relevant chunks and uses them to generate a grounded, accurate answer.

The project must include both a **professional frontend dashboard** and a **complete backend** (processing, embeddings, vector search, RAG, response generation). The final result should feel like a real AI product — not a basic chatbot demo.

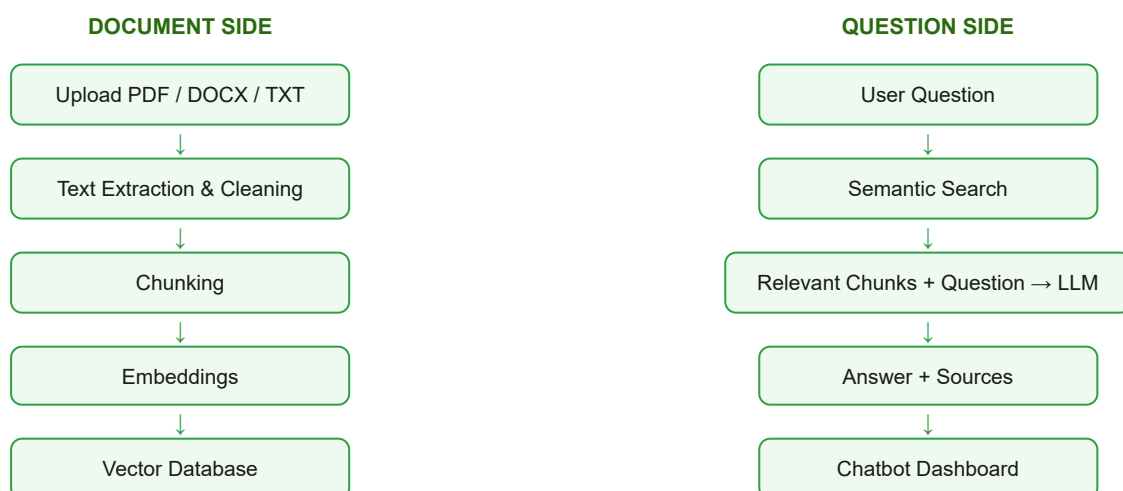
Deadline: Saturday, 12 September 2026. Late submissions receive a negative mark — no late submissions will be accepted.

1. Document Set — Real Estate Sample PDFs

For this cohort: since a live client knowledge base isn't available, generate your own test document set. Use **Claude** to draft **4–5 different sample PDFs** for a fictional real-estate company — for example: *Company Overview*, *Property Listings*, *Services & Fees*, *FAQs*, and *Policies/Terms*. Export them as real PDF files and use them as your upload/testing set for chunking, embedding, and retrieval.

Keep the content internally consistent (same company name, same property/pricing details) across all 4–5 documents — this is what lets you test cross-document questions and catch hallucinations later.

2. Expected Flow



Each stored chunk should carry metadata: document name, document ID, page number (where available), chunk ID.

3. Frontend / Dashboard

Build a professional, responsive dashboard including:

- **Dashboard/Home:** total documents, processed / processing / failed counts, total indexed chunks, total questions/chats — ideally sourced from the backend, not hardcoded.
- **Document Management:** upload, view list with processing status, view document info, delete, re-process. Minimum formats: PDF, DOCX, TXT.
- **Chatbot Interface:** new chat, chat history, user/AI messages with timestamps, loading state, error state, input field + send button, and a source/reference section — clean, responsive, easy to use.

4. Backend

Build a proper backend API. **Python + FastAPI is recommended** (Flask/Django/Express/NestJS also acceptable). It must handle: document upload, text extraction, processing & chunking, embedding generation, vector DB operations, semantic retrieval, RAG logic, LLM communication, chat sessions, document management, and error handling.

Vector DB: ChromaDB / FAISS / Qdrant / pgvector / Weaviate

LLM: Gemini / OpenAI / Claude / Ollama / Hugging Face / local models

5. RAG Requirements & Hallucination Guardrails

- Process documents → chunk → embed → store → retrieve → provide context to the LLM → generate grounded answers.
- The chatbot must **not rely solely on the model's general knowledge**.

If the requested information does not exist in the uploaded documents, the chatbot must clearly say the information could not be found — it must **not confidently fabricate an answer**.

6. Source References

Display the source used to generate every answer, whenever possible:

Q: What AI automation services does your company provide?

A: DevSynt provides AI automation and software development services.

📄 Company_Overview.pdf — Page 3 · 📄 Services.pdf — Page 2

7. Testing

Test with your **5 real-estate sample documents** and at least **10 questions**, covering: direct-answer questions, questions spanning different parts of a document, multi-document questions, questions with no answer in the knowledge base, and questions designed to test hallucination. Document your test results.

8. Code & Security / Cost

Keep the project organized. API keys/secrets must never be committed — use environment variables (`API_KEY`, `DATABASE_URL`, `MODEL_NAME`) and add `.env` to `.gitignore`. Use free/local tools wherever possible — paid subscriptions are not required.

9. Deliverables

- Working application (frontend + backend), source code on GitHub
- README: overview, features, tech stack, install steps, env vars, how to run FE/BE, RAG architecture, API info, limitations
- Architecture diagram: Document → Processing → Chunking → Embeddings → Vector DB → Retrieval → LLM → Response
- Demo video (3–5 min): dashboard, upload, processing, chatbot, Q&A, sources, document management, hallucination test
- Test report: your 10 test questions and results

10. Evaluation

RAG accuracy, quality of retrieval, answer grounding, frontend UI/UX, backend architecture, document processing, error handling, code quality, documentation, and final presentation/demo. **A good UI with weak RAG is not enough — a strong backend with a poor UI is also not enough.** We evaluate the complete product.

11. Submission & Program Rules

- **GitHub:** push your complete, working source code (frontend + backend) to a GitHub repository.
- **Documentation:** upload a full project overview / **README.md** to the same repo (setup, architecture, env vars, RAG design, limitations).
- **Demo video:** record and include your 3–5 minute demo (linked from the README).
- **LinkedIn:** publish a LinkedIn post about this task, **tag DevSynt's company page**, and add the post link inside your GitHub README. This counts toward your evaluation.
- **Deadline: Saturday, 12 September 2026.** Late submissions receive a negative mark — no late submissions will be accepted. We expect a serious, punctual team.
- If anything in this task is unclear, or feels wrong or unfair, contact your mentor directly and ask — don't guess.

 **Need help?** If you face any issue, confusion, or technical problem during this task, contact your mentor directly. Don't stay blocked — ask for help.

Good luck — build something impressive! 